FEATURE ENGINEERING

NBA Player Longevity Prediction

Project Overview

This notebook focuses on analyzing and preparing an NBA player performance dataset for machine learning. The goal is to transform raw sports statistics into model-ready inputs to forecast player longevity (`target_5yrs`).

Core Tasks:

NBA Player Longevity Prediction: Feature Engineering Pipeline

This notebook implements the data processing, statistical filtering, and feature engineering pipeline required to prepare historical NBA player statistics for machine learning models.

Core Objectives:

1. Target Variable Definition and Class Verification
2. Data Leakage Prevention and Feature Selection
3. Handling Missing Values via Robust Imputation
4. Multicollinearity Filtering using Correlation Matrices
5. Domain-Specific Composite Feature Engineering

Step 1: Set Up and Load Data First

import your libraries and load the NBA player performance dataset.

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# Load your dataset
df = pd.read_csv('nba_players.csv')

# Display basic information to see columns and data types
print(df.info())
print(df.head())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1340 entries, 0 to 1339
Data columns (total 22 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   Unnamed: 0   1340 non-null   int64
 1   name         1340 non-null   object
 2   gp           1340 non-null   int64
 3   min          1340 non-null   float64
 4   pts          1340 non-null   float64
 5   fgm          1340 non-null   float64
 6   fga          1340 non-null   float64
 7   fg           1340 non-null   float64
 8   3p_made      1340 non-null   float64
 9   3pa          1340 non-null   float64
 10  3p           1340 non-null   float64
 11  ftm          1340 non-null   float64
 12  fta          1340 non-null   float64
 13  ft           1340 non-null   float64
 14  oreb         1340 non-null   float64
 15  dreb         1340 non-null   float64
 16  reb          1340 non-null   float64
 17  ast          1340 non-null   float64
 18  stl          1340 non-null   float64
 19  blk          1340 non-null   float64
 20  tov          1340 non-null   float64
 21  target_5yrs  1340 non-null   int64
dtypes: float64(18), int64(3), object(1)
memory usage: 230.4+ KB
None
   Unnamed: 0             name  gp   min  pts  fgm  fga    fg  3p_made  3pa  \
0           0   Brandon Ingram  36  27.4  7.4  2.6  7.6  34.7      0.5  2.1
1           1  Andrew Harrison  35  26.9  7.2  2.0  6.7  29.6      0.7  2.8
2           2   JaKarr Sampson  74  15.3  5.2  2.0  4.7  42.2      0.4  1.7
3           3      Malik Sealy  58  11.6  5.7  2.3  5.5  42.6      0.1  0.5
4           4      Matt Geiger  48  11.5  4.5  1.6  3.0  52.4      0.0  0.1

   ...  fta    ft  oreb  dreb  reb  ast  stl  blk  tov  target_5yrs
0  ...  2.3  69.9   0.7   3.4  4.1  1.9  0.4  0.4  1.3            0
1  ...  3.4  76.5   0.5   2.0  2.4  3.7  1.1  0.5  1.6            0
2  ...  1.3  67.0   0.5   1.7  2.2  1.0  0.5  0.3  1.0            0
3  ...  1.3  68.9   1.0   0.9  1.9  0.8  0.6  0.1  1.0            1
4  ...  1.9  67.4   1.0   1.5  2.5  0.3  0.3  0.4  0.8            1

[5 rows x 22 columns]
```

Step 2: Define Target and Explore Data

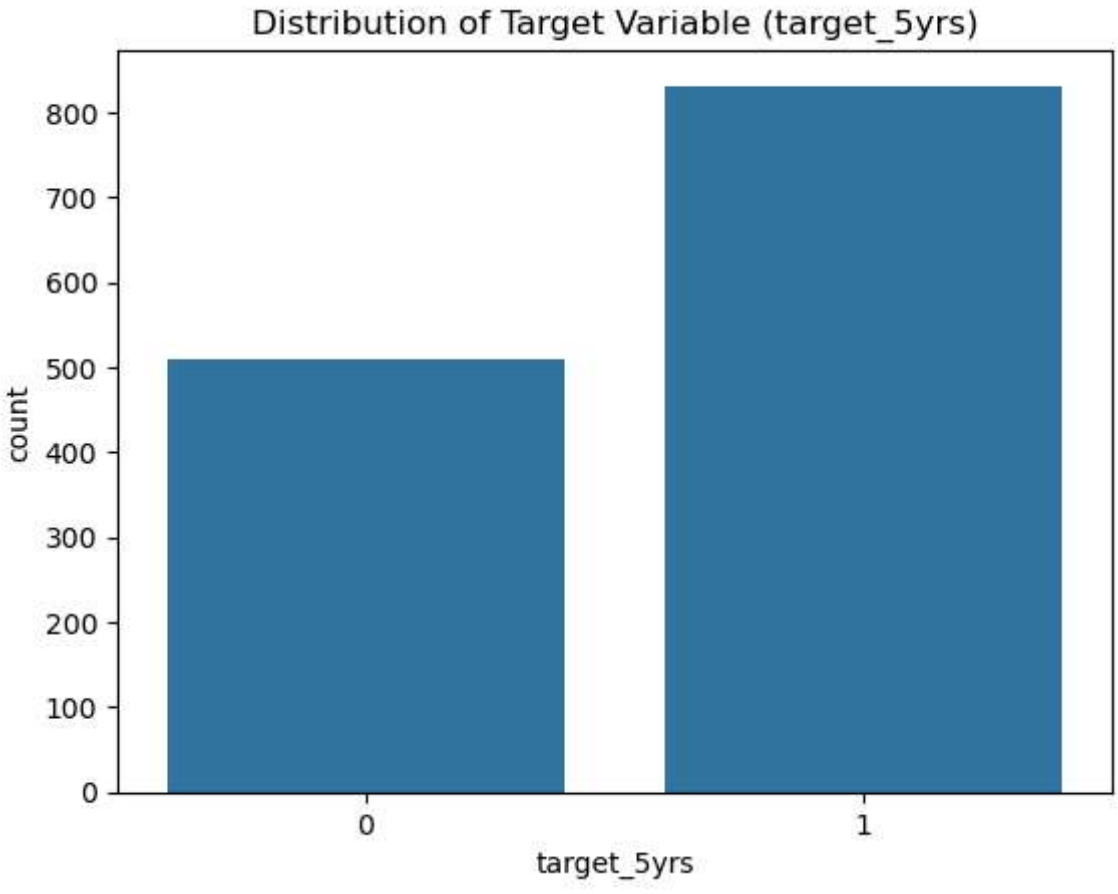
Identify your target variable (target_5yrs) and analyze its distribution to check for class balance.

In [2]:
```
# Verify target variable exists
if 'target_5yrs' in df.columns:
    print("Target variable found.")
else:
    print("Warning: target_5yrs column missing!")

# Check the class distribution
print(df['target_5yrs'].value_counts(normalize=True))

# Visualize the distribution
sns.countplot(x='target_5yrs', data=df)
plt.title('Distribution of Target Variable (target_5yrs)')
plt.show()
```

```
Target variable found.
target_5yrs
1    0.620149
0    0.379851
Name: proportion, dtype: float64
```



Step 3: Remove Non-Predictive Columns

Drop identifiers like player names or IDs that cause data leakage or add noise.

In [3]:
```
# List columns to drop (adjust names based on your df.info() output)
cols_to_drop = ['name', 'player_name', 'ID', 'id', 'team']

# Drop the columns safely
df = df.drop(columns=cols_to_drop, errors='ignore')
print(f"Remaining columns: {df.shape[1]}")
```

```
Remaining columns: 21
```

Step 4: Engineer Composite Features

Create new mathematical metrics that combine existing stats to give the model better predictive signals.

In [4]:
```
print(df.columns.tolist())
```

```
['Unnamed: 0', 'gp', 'min', 'pts', 'fgm', 'fga', 'fg', '3p_made', '3pa', '3p', 'ft
m', 'fta', 'ft', 'oreb', 'dreb', 'reb', 'ast', 'stl', 'blk', 'tov', 'target_5yrs']
```

In [5]:
```
# 1. Points Per Minute (PPM)
# Adding a tiny value (1e-5) prevents division-by-zero errors if min is 0
df['PPM'] = df['pts'] / (df['min'] + 1e-5)

# 2. Composite Efficiency (Example metric based on standard box scores)
# Sum positive contributions and divide by minutes played
df['Composite_Efficiency'] = (df['pts'] + df['reb'] + df['ast'] + df['stl'] + df[

print(df[['PPM', 'Composite_Efficiency']].head())
```

```
        PPM  Composite_Efficiency
0  0.270073              0.518248
1  0.267658              0.553903
2  0.339869              0.601307
3  0.491379              0.784482
4  0.391304              0.695652
```

Step 5: Handle Missing Values

Clean the dataset by replacing any null values with the median of their respective columns to ensure ML-readiness.

In [6]:
```
# Check for missing values before cleaning
print("Missing values per column before:")
print(df.isnull().sum()[df.isnull().sum() > 0])

# Fill missing numerical values with column medians
df = df.fillna(df.median(numeric_only=True))

# Verify zero missing values remain
print("Missing values after cleaning:", df.isnull().sum().sum())
```

```
Missing values per column before:
Series([], dtype: int64)
Missing values after cleaning: 0
```

Step 6: Analyze Correlation & Multicollinearity

Construct a correlation matrix to identify features that are heavily overlapping, which can confuse machine learning models.

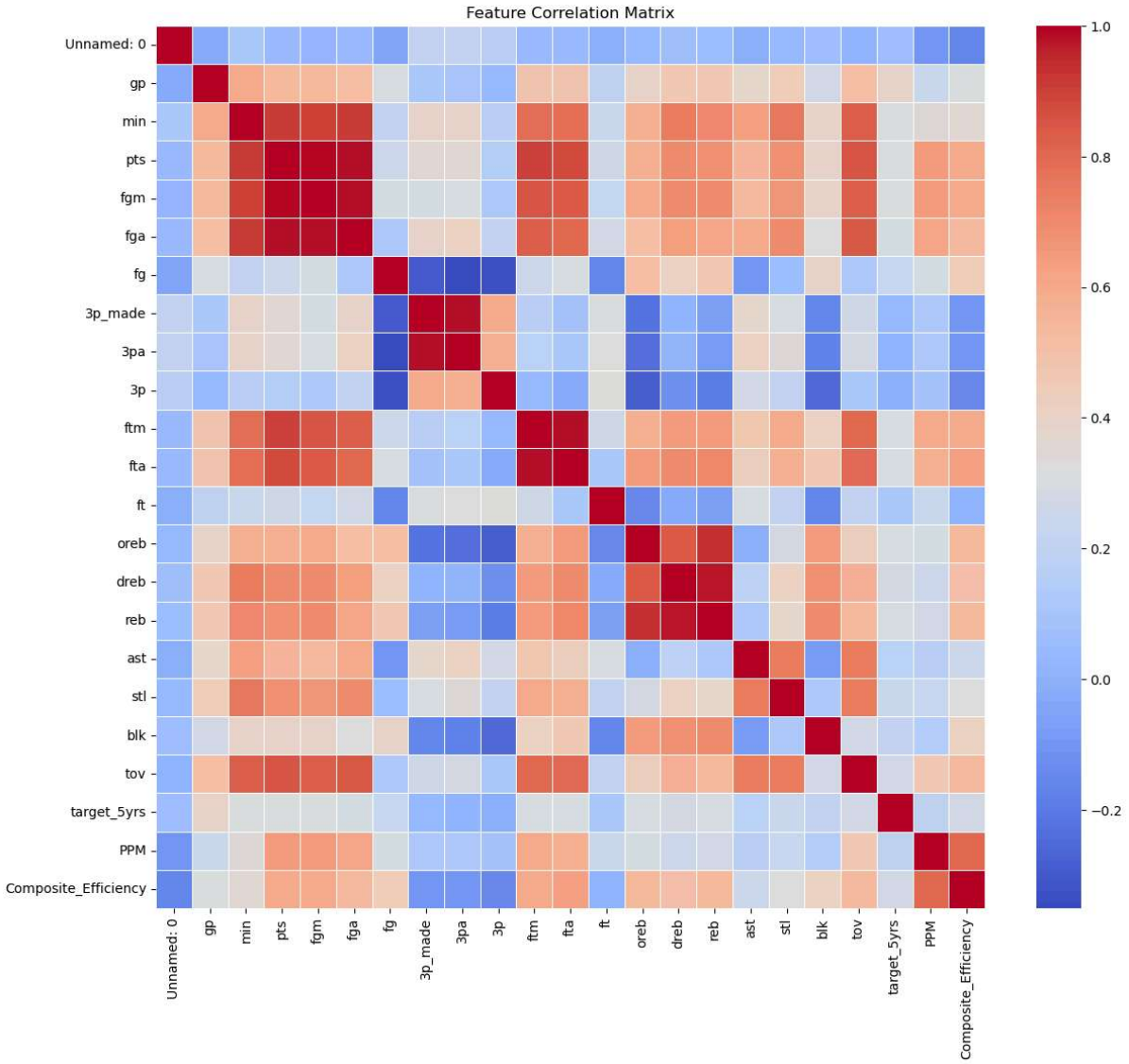
In [7]:
```
# Calculate correlation matrix
corr_matrix = df.corr()

# Plot the heatmap
plt.figure(figsize=(14, 12))
sns.heatmap(corr_matrix, annot=False, cmap='coolwarm', linewidths=0.5)
plt.title('Feature Correlation Matrix')
plt.show()

# Find features with a correlation higher than 0.85 (excluding target)
upper_tri = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astype(bool
to_drop = [column for column in upper_tri.columns if any(upper_tri[column] > 0.85)

print(f"Suggested columns to drop due to high multicollinearity: {to_drop}")

# Drop the redundant features
df = df.drop(columns=to_drop)
```




```
Suggested columns to drop due to high multicollinearity: ['pts', 'fgm', 'fga', '3p
a', 'ftm', 'fta', 'reb', 'tov']
```

Step 7: Final Check and Save

Ensure the data pipeline successfully outputted a clean, structured dataset ready for modeling.

In [8]:
```
# Final dataset shape and preview
print("Final Dataset Shape:", df.shape)
print(df.head())

# Save your clean feature-engineered dataset
df.to_csv('clean_nba_features.csv', index=False)
print("Pipeline complete. Dataset saved!")
```

```
Final Dataset Shape: (1340, 15)
   Unnamed: 0  gp   min    fg  3p_made    3p    ft  oreb  dreb  ast  stl  blk  \
0           0  36  27.4  34.7      0.5  25.0  69.9   0.7   3.4  1.9  0.4  0.4
1           1  35  26.9  29.6      0.7  23.5  76.5   0.5   2.0  3.7  1.1  0.5
2           2  74  15.3  42.2      0.4  24.4  67.0   0.5   1.7  1.0  0.5  0.3
3           3  58  11.6  42.6      0.1  22.6  68.9   1.0   0.9  0.8  0.6  0.1
4           4  48  11.5  52.4      0.0   0.0  67.4   1.0   1.5  0.3  0.3  0.4

   target_5yrs       PPM  Composite_Efficiency
0            0  0.270073              0.518248
1            0  0.267658              0.553903
2            0  0.339869              0.601307
3            1  0.491379              0.784482
4            1  0.391304              0.695652
Pipeline complete. Dataset saved!
```